

DATA ENGINEERING COURSE

JSON & Data Serialization

STUDENT ASSIGNMENT

Instructor: Tanishq Tyagi

Topics Covered

- JSON Fundamentals & JSON vs Python Dictionary
 - Serialization & Deserialization
- `json.dump()`, `json.load()`, `json.dumps()`, `json.loads()`
 - Nested JSON Navigation
- Practical: CSV to JSON Conversion Pipeline

Student Instructions

Course: Data Engineering Course | **Topic:** JSON & Data Serialization | **Instructor:** Tanishq Tyagi

Please read all instructions carefully before attempting the assignment.

- Attempt all questions. Do not leave any question unanswered.
- Write clean, properly indented Python code following PEP 8 standards.
- Use meaningful variable names and add comments wherever necessary.
- For code-based questions, include the expected output as a comment at the end of your code.
- Where JSON output is required, use `indent=4` for pretty-printing.
- Test your code before submission — ensure it runs without errors.
- Submit your work as a single Python file or Jupyter Notebook, as instructed by your instructor.

Section A — Conceptual Questions

Answer the following questions in your own words.

Q1.

What does JSON stand for? List three reasons why JSON is the most popular data exchange format in the tech industry today.

Q2.

Explain the difference between a JSON object and a Python dictionary. Provide at least two key differences with examples.

Q3.

What is serialization? What is deserialization? Explain with a real-world analogy. Which Python functions are used for each?

Q4.

State the difference between `json.dump()` and `json.dumps()`. When would you use each one? Provide a one-line code example for each.

Q5.

State the difference between `json.load()` and `json.loads()`. When would you use each one? Provide a one-line code example for each.

Section B — Coding Questions

Write Python code for each of the following questions. Test your code before submission.

Q6.

Write a Python program that creates the following dictionary and saves it to a file called `profile.json` using `json.dump()`. Print a confirmation message after saving.

Dictionary to serialize:

```
{  
    "name": "Your Name",  
    "course": "Data Engineering",  
    "year": 2024,  
    "skills": ["Python", "SQL", "JSON"]  
}
```

Q7.

Write a Python program that reads the `profile.json` file created in Q6 using `json.load()` and prints each key-value pair on a separate line.

Q8.

Write a Python program that converts the following Python dictionary into a JSON string using `json.dumps()` and prints both the JSON string and its data type.

Dictionary to convert:

```
order = {  
    "order_id": "ORD-1234",  
    "product": "Laptop",  
    "quantity": 2,  
    "price": 75000.00  
}
```

Q9.

You receive the following JSON string from an API. Write a Python program that uses `json.loads()` to parse it and then prints the student's name, subject, and marks.

API response string:

```
api_response = '{"student": "Priya", "subject": "Data Engineering", "marks": 92
}'
```

Q10.

Write a Python program that works with the following nested JSON structure saved in a file called company.json. Your program should:

- Load the JSON file.
- Print the company name.
- Print the name and department of every employee.
- Print the city of the company's office.

company.json structure:

```
{
  "company": "TechCorp",
  "office": {
    "city": "Bengaluru",
    "state": "Karnataka"
  },
  "employees": [
    {"id": 1, "name": "Arjun", "department": "Data Engineering"},
    {"id": 2, "name": "Sneha", "department": "Machine Learning"},
    {"id": 3, "name": "Vikram", "department": "Backend Development"}
  ]
}
```

Section C — Practical Assignment (CSV to JSON Pipeline)

This is the main practical exercise for this session. Complete all 7 steps as a single end-to-end program.

Q11.

You are given the following student marks data in CSV format. Build a complete Python data pipeline that processes this data and produces a final JSON output file.

Input Data (CSV):

students.csv — Note: some mark fields are intentionally left blank.

```
id,name,maths,science,english
1,Rahul,85,90,88
2,Amit,78,,80
3,Neha,95,92,96
4,Priya,,85,89
5,Karan,70,75,
```

Your pipeline must implement the following steps in order:

Step 1	Read the data from the CSV file using Python's csv module.
Step 2	Replace any missing marks with 0 (blank cells should be treated as 0).
Step 3	Convert all mark values (maths, science, english) from strings to integers.
Step 4	Calculate the total marks for each student (maths + science + english). Maximum total = 300.
Step 5	Calculate the percentage for each student: $(total / 300) \times 100$. Round to 2 decimal places.
Step 6	Determine the grade for each student based on the following criteria: • Grade A: Percentage ≥ 90 • Grade B: Percentage ≥ 75 • Grade C: Percentage ≥ 60 • Grade D: Percentage < 60
Step 7	Create and save the final processed data as a JSON file called student_results.json. Use indent=4 for pretty-printing.

Expected Output Format (student_results.json):

Each student record in the JSON output should contain the following fields:

- id, name, maths, science, english (original fields)
- total (calculated total marks)
- percentage (calculated percentage, rounded to 2 decimal places)
- grade (A, B, C, or D based on the grading criteria above)

Section D — Challenge Questions

These questions go beyond the classroom material. Attempt them for additional learning.

Q12.

Modify your CSV-to-JSON pipeline from Q11 to also include a summary section at the end of the JSON file containing:

- Total number of students processed.
- Class average percentage (rounded to 2 decimal places).
- Highest scoring student name and their percentage.
- Lowest scoring student name and their percentage.
- Grade distribution: how many students received each grade (A, B, C, D).

Q13.

Write a Python program that reads a nested JSON file containing flight data (similar to the Cleartrip example from class). Your program should:

- Load the JSON file.
- Extract and print the airport codes from the nearbyAirportFilter array.
- Print the total number of airports found.
- Handle the case where the key does not exist using a try-except block or .get() with a default value.

Q14.

You are given a list of Python dictionaries representing product orders. Write a Python program that:

- Serializes the list to a JSON file called orders.json.
- Reads it back and calculates the total revenue (price × quantity for each order).
- Adds a 'revenue' field to each order dictionary.
- Saves the updated data back to orders.json.
- Prints the grand total revenue at the end.

Input data for Q14:

```
orders = [  
    {"order_id": "A001", "product": "Laptop", "quantity": 2, "price": 75000},
```

```
{  
  "order_id": "A002", "product": "Mouse",    "quantity": 5, "price": 500},  
  {"order_id": "A003", "product": "Keyboard", "quantity": 3, "price": 1200},  
}
```